

Leyan Li

Leyanpower@gmail.com | +86 18805506898 | GitHub: <https://github.com/lionleepower>

Education:

University of Edinburgh United Kingdom | Sep 2025 – Sep 2026 Expected

MSc High Performance Computing with Data Science

University of Liverpool United Kingdom | Sep 2023 – Jun 2025

BSc Computer Science, Xi'an Jiaotong-Liverpool University 2+2 Programme, First Class Honours

Technical Skills

Programming and Engineering: Proficient in Python and C/C++, with experience in AI application prototyping, backend API development, data processing, log parsing, experiment automation, model training workflows, and performance analysis scripts. Familiar with SQL, JavaScript, HTML, and CSS.

AI Agent and LLM Applications: Familiar with LLM Agent architectures, including tool calling, task decomposition, RAG, prompt engineering, memory management, multi-turn dialogue state management, and Markdown report generation. Experienced in integrating LLM capabilities with external tools, file systems, databases, caches, and automation scripts.

RAG and Vector Retrieval: Familiar with Markdown document parsing, text chunking, embedding generation, FAISS / Chroma vector indexing, top-k semantic retrieval, and context construction for retrieval-augmented question answering and report generation.

Backend and System Design: Familiar with FastAPI, REST APIs, HTTP, client-server interaction, request validation, exception handling, logging, TracelID, task scheduling, rate limiting, idempotency, and observability concepts.

Databases and Caching: Familiar with MySQL / SQLite, including indexing, transactions, locks, and query optimization. Familiar with Redis use cases such as hot query caching, short-term session state, TTL, cache penetration, cache breakdown, and cache consistency.

Machine Learning and Data Analysis: Proficient in PyTorch, with experience in deep learning and multi-agent reinforcement learning experiments. Familiar with Pandas, NumPy, and Matplotlib for structured data processing, metric calculation, experimental log analysis, and visualization.

Tools and Development Environment: Familiar with Linux, Bash, Git, VS Code, and basic Docker operations. Experienced in using automation scripts to manage experiments, process logs, maintain project documentation, and build reproducible development workflows.

Projects

AI Agent System for HPC Benchmark Performance Analysis

Project Description: Built an AI Agent system for high-performance computing benchmark analysis, designed to assist with PETSc benchmark management, Slurm / PETSc log parsing, historical performance data querying, HPC knowledge base retrieval, and Markdown performance report generation. The system integrates Agent tool calling, RAG retrieval, database storage, Redis caching, backend APIs, and observability mechanisms, allowing users to query experiment results, compare parallel configurations, analyze performance bottlenecks, and generate reports through natural language interaction.

Tech Stack: Python, FastAPI, LangChain / LlamaIndex, FAISS / Chroma, OpenAI API / Local LLM, SQLite / MySQL, Redis, Pandas, Matplotlib, PETSc, MPI, OpenMP, Slurm, Docker, Linux

Responsibilities:

- Designed the overall AI Agent architecture, dividing the system into API layer, Agent planning layer, tool calling layer, RAG retrieval layer, database layer, cache layer, report generation layer, and observability module to support multi-step performance analysis workflows.
- Built backend services with FastAPI, providing APIs for natural language question answering, experiment result querying, log upload and parsing, history retrieval, performance metric calculation, and Markdown report generation, with unified request validation, exception handling, and response formatting.
- Designed an Agent tool calling workflow that allows the system to automatically select database query, log parsing, metric calculation, knowledge retrieval, chart generation, and report generation tools according to the user's question.
- Built a RAG retrieval pipeline by indexing PETSc, MPI, OpenMP, Slurm, Linaro MAP, and personal HPC experiment notes, including Markdown document parsing, text chunking, embedding generation, vector index construction, and top-k semantic retrieval.

- Designed database schemas for storing benchmark configurations, execution status, performance metrics, error logs, and analysis records, supporting queries by problem size, execution mode, total cores, and creation time.
 - Designed indexing strategies for common query patterns, such as finding the best configuration by problem size and core count, comparing MPI-only and hybrid MPI+OpenMP execution modes, and tracking experiment history by time range.
 - Introduced Redis caching for hot natural language queries, frequently used experiment statistics, short-term session state, and query-hash-based analysis results, using TTL to control cache lifecycle and reduce repeated database queries, vector retrieval, and LLM calls.
 - Implemented Slurm / PETSc log parsing modules to extract problem size, nodes, MPI ranks, OpenMP threads, total cores, runtime, execution mode, solver status, and error messages from raw outputs and convert them into structured experiment records.
 - Implemented performance metric calculation and visualization modules with Pandas and Matplotlib, supporting automatic calculation of runtime, speedup, and parallel efficiency, as well as comparison charts across problem sizes and parallel configurations.
 - Generated a TraceID for each Agent request and recorded user questions, tool invocation chains, retrieved documents, cache hits, database query latency, RAG retrieval results, and LLM response latency for debugging retrieval failures, tool errors, and API performance issues.
 - Designed a report generation workflow that integrates experiment data, plots, RAG retrieval results, and Agent-generated analysis into Markdown performance reports that can be stored in Obsidian or GitHub documentation.
 - Validated the system with real PETSc benchmark data, supporting analysis of 2D / 3D stencil problems under MPI-only and hybrid MPI+OpenMP configurations, including runtime, speedup, parallel efficiency, and performance bottleneck comparison.
-

Value Function Decomposition in Multi-Agent Actor-Critic Methods

Project Description: Researched the integration of value function decomposition methods, including VDN and QMIX, into the MAPPO framework to investigate the impact of factorised critics on cooperation efficiency, convergence stability, and policy performance in cooperative multi-agent environments. The project demonstrates experience in multi-agent cooperation, task decomposition, policy learning, PyTorch model implementation, experiment evaluation, and result analysis, serving as an algorithmic foundation for AI Agent and multi-agent collaboration scenarios.

Tech Stack: Python, PyTorch, ePyMARL, PyMARL, MAPPO, VDN, QMIX, Multi-Agent Reinforcement Learning

Responsibilities:

- Implemented MAPPO, MAPPO-VDN, and MAPPO-QMIX variants based on the ePyMARL / PyMARL framework, covering training configuration, experiment execution, and result evaluation.
 - Modified the critic architecture in the actor-critic framework by introducing VDN-style additive value decomposition and QMIX-style monotonic mixing networks to explore the applicability of factorised critics in cooperative multi-agent tasks.
 - Supported the centralized training with decentralized execution paradigm, using global state and joint action information during training while allowing each agent to make independent decisions based on local observations during execution.
 - Designed and conducted comparative experiments in Matrix Game and Predator-Prey environments to evaluate different algorithms in terms of test return, convergence speed, final return, and cooperative performance.
 - Built experiment logging and result analysis workflows to visualize and compare reward curves, test return, convergence behaviour, and training stability.
 - Analyzed instability issues when applying value decomposition methods to actor-critic frameworks, including critic approximation error, multi-agent non-stationarity, credit assignment, and the representational limitations of mixing networks.
 - Completed a full academic paper covering algorithm design, implementation details, experimental results, limitations, and future improvement directions.
-

Publication

Li, Leyan. *Convolutional Neural Networks Based Medical Image Analysis*. International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.